

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

ASSESSMENT TOOL 1 – Comparative Analysis

Course Code:	CSA03	Course Title:	Data Structures
CO Assessed:	CO4 –Articulation (BL3)	Assessment:	Assessment Tool 1 – Comparative Analysis
Weightage:	20% of CIA	Total Marks:	50
CO3 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data		
Student Name:	T.VAISHNAVI	Reg. No.:	192421370
Section / Batch:	B-TECH(IT)	Date:	13/08/2026

Code of Conduct

I T.VAISHNAVI(192421370) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Instructions

- This test contains 5 Coding challenge Total: 50 Marks.
- Evaluation of Answer Quality -Checking whether the answer includes: Scope of Items, Comparison Depth, Use of Evidence, Critical Thinking and Clarity of Presentation
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing - Priority Queue & Heaps-	Hashing, Heap	1,2	20
Sorting Algorithm	Insertion sort, Heap sort	3,4	20
Parallel Sorting algorithms-Time complexity.	Merge Sort, Quick sort	5	10
GRAND TOTAL:		50Marks	

Annexure A – Experiments

Q1) A system needs to store large data and perform frequent search operations. For this, compare Hashing and Sorting-based search (Binary Search) in terms of:

- Time complexity
- Space usage
- Update operations
- Real-world applicability

Conclude which is better for dynamic systems.

```
1 #include <stdio.h>
2
3 #define SIZE 10
4
5 int hashTable[SIZE];
6
7 void initHash()
8 {
9     int i;
10    for (i = 0; i < SIZE; i++)
11        hashTable[i] = -1;
12 }
13
14 void insertHash(int key)
15 {
16     int index = key % SIZE;
17
18     while (hashTable[index] != -1)
19         index = (index + 1) % SIZE;
20
21     hashTable[index] = key;
22 }
23
24 int searchHash(int key)
25 {
26     int index = key % SIZE;
27     int start = index;
28
29     while (hashTable[index] != -1)
30     {
31         if (hashTable[index] == key)
32             return index;
33
34         index = (index + 1) % SIZE;
35
36         if (index == start)
37             break;
38     }
39
40     return -1;
41 }
```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.78 Mb

Sample Input

7
10 20 30 40 50 60 70
50

Your Output

Enter number of elements (max 10): Enter 7 sorted elements:
Enter element to search: Hashing: 50 found at hash index 4
Binary Search: 50 found at array index 4

```
12 }
13
14 void insertHash(int key)
15 {
16     int index = key % SIZE;
17
18     while (hashTable[index] != -1)
19         index = (index + 1) % SIZE;
20
21     hashTable[index] = key;
22 }
23
24 int searchHash(int key)
25 {
26     int index = key % SIZE;
27     int start = index;
28
29     while (hashTable[index] != -1)
30     {
31         if (hashTable[index] == key)
32             return index;
33
34         index = (index + 1) % SIZE;
35
36         if (index == start)
37             break;
38     }
39
40     return -1;
41 }
42
43 int binarySearch(int a[], int n, int key)
44 {
45     int low = 0;
46     int high = n - 1;
47     int mid;
48
49     while (low <= high)
50     {
51         mid = (low + high) / 2;
52
53         if (a[mid] == key)
54             return mid;
55
56         if (a[mid] < key)
```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.78 Mb

Sample Input

7
10 20 30 40 50 60 70
50

Your Output

Enter number of elements (max 10): Enter 7 sorted elements:
Enter element to search: Hashing: 50 found at hash index 4
Binary Search: 50 found at array index 4

```

70 int h, b;
71 initHash();
72
73 printf("Enter number of elements (max 10): ");
74 scanf("%d", &n);
75
76 printf("Enter %d sorted elements:\n", n);
77
78 for (i = 0; i < n; i++)
79 {
80     scanf("%d", &a[i]);
81     insertHash(a[i]);
82 }
83
84 printf("Enter element to search: ");
85 scanf("%d", &key);
86
87 h = searchHash(key);
88 b = binarySearch(a, n, key);
89
90 if (h != -1)
91     printf("Hashing: %d found at hash index %d\n", key, h);
92 else
93     printf("Hashing: %d not found\n", key);
94
95 if (b != -1)
96     printf("Binary Search: %d found at array index %d\n", key, b);
97 else
98     printf("Binary Search: %d not found\n", key);
99
100 return 0;
101 }
102

```

Time: 0.0000 secs Memory: 1.78 Mb

Sample Input

```

7
10 20 30 40 50 60 70
50

```

Your Output

```

Enter number of elements (max 10): Enter 7 sorted elements:
Enter element to search: Hashing: 50 found at hash index 4
Binary Search: 50 found at array index 4

```

Q2) Compare Linear Probing and Separate Chaining for collision handling in hashing based on:

- Clustering effect
- Memory usage
- Performance under high load factor

Which method is more suitable for large-scale applications and why?

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define SIZE 10
5
6 int linearTable[SIZE];
7
8 struct Node {
9     int data;
10    struct Node *next;
11 };
12
13 struct Node *chainTable[SIZE];
14
15 void init()
16 {
17     int i;
18     for (i = 0; i < SIZE; i++)
19     {
20         linearTable[i] = -1;
21         chainTable[i] = NULL;
22     }
23 }
24
25
26 void linearInsert(int key)
27 {
28     int index = key % SIZE;
29
30     while (linearTable[index] != -1)
31         index = (index + 1) % SIZE;
32
33     linearTable[index] = key;
34 }

```

Run

7
10 20 30 40 50 60 70

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.88 Mb

Sample Input

```

7
10 20 30 40 50 60 70

```

Your Output

```

Enter number of elements: Enter elements:

Linear Probing:
Index 0: 10

```

```

32     linearTable[index] = key;
33 }
34 }
35 void chainInsert(int key)
36 {
37     int index = key % SIZE;
38
39     struct Node *newNode;
40     newNode = (struct Node *)malloc(sizeof(struct Node));
41
42     newNode->data = key;
43     newNode->next = chainTable[index];
44     chainTable[index] = newNode;
45 }
46
47 void displayLinear()
48 {
49     int i;
50
51     printf("\nLinear Probing:\n");
52     for (i = 0; i < SIZE; i++)
53     {
54         printf("Index %d: ", i);
55         if (linearTable[i] == -1)
56             printf("Empty");
57         else
58             printf("%d", linearTable[i]);
59         printf("\n");
60     }
61 }
62
63 void displayChain()
64 {
65     int i;
66     struct Node *temp;
67
68     printf("\nSeparate Chaining:\n");
69     for (i = 0; i < SIZE; i++)
70     {
71         if (chainTable[i] != NULL)
72             printf("Index %d: ", i);
73         temp = chainTable[i];
74         while (temp != NULL)
75         {
76             printf("%d", temp->data);
77             if (temp->next != NULL)
78                 printf(" -> ");
79             temp = temp->next;
80         }
81         printf("\n");
82     }
83 }
84
85 int main()
86 {
87     int n, key, i;
88     init();
89     printf("Enter number of elements: ");
90     scanf("%d", &n);
91     printf("Enter elements:\n");
92     for (i = 0; i < n; i++)
93     {
94         scanf("%d", &key);
95         linearInsert(key);
96         chainInsert(key);
97     }
98     displayLinear();
99     displayChain();
100     return 0;
101 }

```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.88 Mb

Sample Input

7
10 20 30 40 50 60 70

Your Output

Index 6: 70
Index 7: Empty
Index 8: Empty
Index 9: Empty

Separate Chaining:
Index 0: 70 -> 60 -> 50 -> 40 -> 30 -> 20 -> 10
Index 1: Empty
Index 2: Empty
Index 3: Empty

```

83     }
84     else
85     {
86         while (temp != NULL)
87         {
88             printf("%d", temp->data);
89             if (temp->next != NULL)
90                 printf(" -> ");
91             temp = temp->next;
92         }
93         printf("\n");
94     }
95 }
96
97 int main()
98 {
99     int n, key, i;
100     init();
101     printf("Enter number of elements: ");
102     scanf("%d", &n);
103     printf("Enter elements:\n");
104     for (i = 0; i < n; i++)
105     {
106         scanf("%d", &key);
107         linearInsert(key);
108         chainInsert(key);
109     }
110     displayLinear();
111     displayChain();
112     return 0;
113 }

```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.88 Mb

Sample Input

7
10 20 30 40 50 60 70

Your Output

Index 1: Empty
Index 2: Empty
Index 3: Empty
Index 4: Empty
Index 5: Empty
Index 6: Empty
Index 7: Empty
Index 8: Empty
Index 9: Empty

Q3) Compare Sequential Sorting and Parallel Sorting (Multi-threading / MapReduce) in terms of:

- Execution time
- Scalability
- Overhead
- Suitability for small vs large datasets

Explain why parallel sorting is not always the best choice

Online C Compiler

```

1 #include <stdio.h>
2 #include <pthread.h>
3 #include <time.h>
4 #define MAX 100
5
6 int a[MAX], b[MAX];
7 int n;
8
9 typedef struct {
10     int start;
11     int end;
12 } ThreadData;
13
14 void sequentialSort(int arr[], int n)
15 {
16     int i, j, temp;
17     for (i = 0; i < n - 1; i++)
18     {
19         for (j = i + 1; j < n; j++)
20         {
21             if (arr[i] > arr[j])
22             {
23                 temp = arr[i];
24                 arr[i] = arr[j];
25                 arr[j] = temp;
26             }
27         }
28     }
29 }
30
31 void *parallelSort(void *arg)
32 {
33     ThreadData *data = (ThreadData *)arg;
34     int i, j, temp;
35     for (i = data->start; i < data->end - 1; i++)
36     {
37         for (j = i + 1; j < data->end; j++)
38         {
39             if (arr[i] > arr[j])
40             {
41                 temp = arr[i];
42                 arr[i] = arr[j];
43                 arr[j] = temp;
44             }
45         }
46     }
47 }

```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.88 Mb

Sample Input

8
64 25 12 22 11 90 45 30

Your Output

11 12 22 25 30 45 64 90
Parallel Sorting Result:
11 12 22 25 30 45 64 90

Sequential Sorting Time: 0.000001 seconds
Parallel Sorting Time: 0.000004 seconds

```
21- {
22-     for (j = 0; j < n - i - 1; j++)
23-     {
24-         if (arr[j] > arr[j + 1])
25-         {
26-             temp = arr[j];
27-             arr[j] = arr[j + 1];
28-             arr[j + 1] = temp;
29-         }
30-     }
31- }
32- }
33-
34- void *parallelSort(void *arg)
35- {
36-     ThreadData *data = (ThreadData *)arg;
37-     int i, j, temp;
38-     for (i = data->start; i < data->end - 1; i++)
39-     {
40-         for (j = data->start; j < data->end - 1 - (i - data->start); j++)
41-         {
42-             if (b[j] > b[j + 1])
43-             {
44-                 temp = b[j];
45-                 b[j] = b[j + 1];
46-                 b[j + 1] = temp;
47-             }
48-         }
49-     }
50- }
51-
52- return NULL;
53- }
54-
55- void merge()
56- {
57-     int mid = n / 2;
58-     int i = 0, j = mid, k = 0;
59-     int temp[MAX];
60-     while (i < mid && j < n)
61-     {
62-         if (b[i] < b[j])
63-             temp[k++] = b[i++];
64-         else
65-             temp[k++] = b[j++];
66-     }
67- }
```

64 25 12 22 11 90 45 30

Output

Status: Successfully executed

Time: 0.0000 secs Memory: 1.888 Mb

Sample Input

8
64 25 12 22 11 90 45 30

Your Output

Sequential Sorting Result:
11 12 22 25 30 45 64 90
Parallel Sorting Result:
11 12 22 25 30 45 64 90

Sequential Sorting Time: 0.0000001 seconds
Parallel Sorting Time: 0.000064 seconds

Parallel sorting divides the data into multiple parts and sorts them simultaneously using threads.

```
107- //end of mergeSort
108- clock_t start2 = clock();
109-
110- pthread_create(&t1, NULL, parallelSort, &d1);
111- pthread_create(&t2, NULL, parallelSort, &d2);
112-
113- pthread_join(t1, NULL);
114- pthread_join(t2, NULL);
115-
116- merge();
117-
118- clock_t end2 = clock();
119-
120- printf("\nSequential Sorting Result:\n");
121-
122- for (i = 0; i < n; i++)
123-     printf("%d ", a[i]);
124-
125- printf("\n");
126-
127- printf("\nParallel Sorting Result:\n");
128-
129- for (i = 0; i < n; i++)
130-     printf("%d ", b[i]);
131-
132- printf("\n");
133-
134- printf("\nSequential Sorting Time: %.6f seconds\n",
135-        (double)(end1 - start1) / CLOCKS_PER_SEC);
136-
137- printf("\nParallel Sorting Time: %.6f seconds\n",
138-        (double)(end2 - start2) / CLOCKS_PER_SEC);
139-
140- printf("\nParallel sorting divides the data into multiple parts\n");
141- printf("and sorts them simultaneously using threads.\n");
142-
143- return 0;
144- }
```

Output

Status: Successfully executed

Time: 0.0000 secs Memory: 1.888 Mb

Sample Input

8
64 25 12 22 11 90 45 30

Your Output

Sequential Sorting Result:
11 12 22 25 30 45 64 90
Parallel Sorting Result:
11 12 22 25 30 45 64 90

Sequential Sorting Time: 0.0000001 seconds
Parallel Sorting Time: 0.000064 seconds

Parallel sorting divides the data into multiple parts and sorts them simultaneously using threads.

Q4) Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays?

Online C Compiler Practice C problems from our road

C

```
1 #include <stdio.h>
2 #include <time.h>
3
4 #define MAX 100
5
6 void swap(int *a, int *b)
7 {
8     int temp = *a;
9     *a = *b;
10    *b = temp;
11 }
12
13 /* Quick Sort */
14 int partition(int a[], int low, int high)
15 {
16     int pivot = a[high];
17     int i = low - 1;
18     int j;
19
20     for (j = low; j < high; j++)
21     {
22         if (a[j] < pivot)
23         {
24             i++;
25             swap(&a[i], &a[j]);
26         }
27     }
28
29     swap(&a[i + 1], &a[high]);
30
31     return i + 1;
32 }
33
```

Run

8
64 25 12 22 11 90 45 30

Output

Status: Successfully executed

Time: 0.0000 secs Memory: 1.804 Mb

Sample Input

8
64 25 12 22 11 90 45 30

Your Output

Enter number of elements: Enter elements:
Quick Sort Result:
11 12 22 25 30 45 64 90

```

52     while (i <= mid && j <= high)
53     {
54         if (a[i] <= a[j])
55             temp[k++] = a[i++];
56         else
57             temp[k++] = a[j++];
58     }
59
60     while (i <= mid)
61         temp[k++] = a[i++];
62
63     while (j <= high)
64         temp[k++] = a[j++];
65
66     for (i = low; i <= high; i++)
67         a[i] = temp[i];
68 }
69
70 void mergeSort(int a[], int low, int high)
71 {
72     if (low < high)
73     {
74         int mid = (low + high) / 2;
75
76         mergeSort(a, low, mid);
77         mergeSort(a, mid + 1, high);
78
79         merge(a, low, mid, high);
80     }
81 }
82
83 void display(int a[], int n)
84 {
85     int i;
86
87     for (i = 0; i < n; i++)
88         printf("%d ", a[i]);
89
90     printf("\n");
91 }
92
93 int main()
94 {
95     int a[MAX], b[MAX];

```

```

8
64 25 12 22 11 90 45 30

```

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.804 Mb

Sample Input

```

8
64 25 12 22 11 90 45 30

```

Your Output

Quick Sort Result:
11 12 22 25 30 45 64 90
Quick Sort Time: 0.000001 seconds

Merge Sort Result:
11 12 22 25 30 45 64 90
Merge Sort Time: 0.000001 seconds

Conclusion:
Merge Sort is preferable for linked lists because it

```

102     printf("Enter elements:\n");
103     for (i = 0; i < n; i++)
104     {
105         scanf("%d", &a[i]);
106         b[i] = a[i];
107     }
108
109     start = clock();
110     quickSort(a, 0, n - 1);
111     end = clock();
112
113     printf("\nQuick Sort Result:\n");
114     display(a, n);
115
116     printf("Quick Sort Time: %.6f seconds\n",
117           (double)(end - start) / CLOCKS_PER_SEC);
118
119     start = clock();
120     mergeSort(b, 0, n - 1);
121     end = clock();
122
123     printf("\nMerge Sort Result:\n");
124     display(b, n);
125
126     printf("Merge Sort Time: %.6f seconds\n",
127           (double)(end - start) / CLOCKS_PER_SEC);
128
129     printf("\nConclusion:\n");
130     printf("Merge Sort is preferable for linked lists because it\n");
131     printf("works efficiently with sequential access and guarantees O(n log n).\n");
132     printf("Quick Sort is often preferable for arrays because of good cache locality\n");
133     );
134     printf("and in-place sorting.\n");
135
136     return 0;
137 }
138

```

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.804 Mb

Sample Input

```

8
64 25 12 22 11 90 45 30

```

Your Output

Merge Sort Result:
11 12 22 25 30 45 64 90
Merge Sort Time: 0.000001 seconds

Conclusion:
Merge Sort is preferable for linked lists because it
works efficiently with sequential access and guarantees $O(n \log n)$.
Quick Sort is often preferable for arrays because of good cache locality
and in-place sorting.

Q5) Compare heap sort versus quick sort for sorting 1 billion integers in a real-time system with strict latency requirements (no operation can exceed 1 second). Analyze worst-case time complexity, space complexity, and predictability. Which algorithm guarantees $O(n \log n)$ worst-case and why would you choose it despite quick sort's faster average case?

```

C
1 #include <stdio.h>
2 #include <time.h>
3
4 #define MAX 100
5
6 void swap(int *a, int *b)
7 {
8     int temp = *a;
9     *a = *b;
10    *b = temp;
11 }
12
13 /* Heap Sort */
14 void heapify(int a[], int n, int i)
15 {
16     int largest = i;
17     int left = 2 * i + 1;
18     int right = 2 * i + 2;
19
20     if (left < n && a[left] > a[largest])
21         largest = left;
22
23     if (right < n && a[right] > a[largest])
24         largest = right;
25
26     if (largest != i)
27     {
28         swap(&a[i], &a[largest]);
29         heapify(a, n, largest);
30     }
31 }
32
33 void heapSort(int a[], int n)
34 {
35     int i;
36     for (i = n / 2 - 1; i >= 0; i--)
37

```

Run

8
64 25 12 22 11 90 45 30

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.856 Mb

Sample Input

8
64 25 12 22 11 90 45 30

Your Output

Heap Sort Result:
11 12 22 25 30 45 64 90
Heap Sort Time: 0.000001 seconds

Quick Sort Result:
11 12 22 25 30 45 64 90
Quick Sort Time: 0.000001 seconds

```

42     swap(&a[0], &a[1]);
43     heapify(a, i, 0);
44 }
45 }
46
47 /* Quick Sort */
48 int partition(int a[], int low, int high)
49 {
50     int pivot = a[high];
51     int i = low - 1;
52     int j;
53
54     for (j = low; j < high; j++)
55     {
56         if (a[j] < pivot)
57         {
58             i++;
59             swap(&a[i], &a[j]);
60         }
61     }
62     swap(&a[i + 1], &a[high]);
63     return i + 1;
64 }
65
66 void quickSort(int a[], int low, int high)
67 {
68     if (low < high)
69     {
70         int p = partition(a, low, high);
71         quickSort(a, low, p - 1);
72         quickSort(a, p + 1, high);
73     }
74 }
75
76 void display(int a[], int n)
77 {
78     int i;
79     for (i = 0; i < n; i++)
80         printf("%d ", a[i]);
81 }
82
83
84

```

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.856 Mb

Sample Input

8
64 25 12 22 11 90 45 30

Your Output

Quick Sort Result:
11 12 22 25 30 45 64 90
Quick Sort Time: 0.000001 seconds

Complexity:
Heap Sort Worst Case: $O(n \log n)$
Quick Sort Worst Case: $O(n^2)$

Conclusion:
Heap Sort is preferred for strict real-time systems
Because its worst-case $O(n \log n)$ worst-case time


```

102     scanf("%d", &a[i]);
103     b[i] = a[i];
104 }
105
106 /* Heap Sort */
107 start = clock();
108
109 heapSort(a, n);
110
111 end = clock();
112
113 printf("\nHeap Sort Result:\n");
114 display(a, n);
115
116 printf("Heap Sort Time: %.6f seconds\n",
117        (double)(end - start) / CLOCKS_PER_SEC);
118
119 /* Quick Sort */
120 start = clock();
121
122 quickSort(b, 0, n - 1);
123
124 end = clock();
125
126 printf("\nQuick Sort Result:\n");
127 display(b, n);
128
129 printf("Quick Sort Time: %.6f seconds\n",
130        (double)(end - start) / CLOCKS_PER_SEC);
131
132 printf("\nComplexity:\n");
133 printf("Heap Sort Worst Case: O(n log n)\n");
134 printf("Quick Sort Worst Case: O(n^2)\n");
135
136 printf("\nConclusion:\n");
137 printf("Heap Sort is preferred for strict real-time systems\n");
138 printf("because it guarantees O(n log n) worst-case time.\n");
139
140 return 0;
141 }

```

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.856 Mb

Sample Input

```

8
64 25 12 22 11 90 45 30

```

Your Output

```

11 16 22 25 30 45 64 90
Quick Sort Time: 0.000001 seconds

```

Complexity:

Heap Sort Worst Case: $O(n \log n)$
Quick Sort Worst Case: $O(n^2)$

Conclusion:

Heap Sort is preferred for strict real-time systems
because it guarantees $O(n \log n)$ worst-case time.